SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect	5

Sl. No	Questions	Bloom's Level
	detection and test completeness. Provide a structured explanation of your solution.	
4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5

Sl. No	Questions	Bloom's Level
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5
9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5

Sl. No	Questions	Bloom's Level
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

*I **Mugesh S (192411055)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

ANSWERS

Question 1: Online Banking Fund Transfer Failures

1. Problem Analysis

The banking system experiences failures during fund transfers, especially at minimum and maximum transaction limits. This indicates that the existing test cases do not adequately cover boundary values and invalid inputs. Random testing alone cannot guarantee that important boundary conditions are tested.

2. Equivalence Class Partitioning (ECP)

The transfer amount can be divided into the following equivalence classes. Assume the permitted transfer range is ₹100 to ₹50,000.

Equivalence Class	Input Value	Expected Result
Below minimum	₹99	Reject transfer
Minimum valid	₹100	Accept if other conditions are satisfied
Valid range	₹500	Accept
Maximum valid	₹50,000	Accept if other conditions are satisfied
Above maximum	₹50,001	Reject transfer
Zero amount	₹0	Reject transfer
Negative amount	-₹100	Reject transfer

3. Boundary Value Analysis (BVA)

The following values should be tested around both boundaries:

- Minimum boundary: ₹99, ₹100, ₹101.
- Maximum boundary: ₹49,999, ₹50,000, ₹50,001.

Additional tests should verify insufficient balance, invalid account numbers, network failures, and duplicate transfer requests.

4. Evaluation of Random Testing

Random testing can identify unexpected combinations and unusual failures, but it may repeatedly generate ordinary values without testing critical boundaries. Therefore, random testing should be retained as supplementary testing rather than used as the primary strategy.

5. Improved Testing Strategy

1. Apply ECP to identify valid and invalid input classes.
2. Apply BVA to test values immediately around transaction limits.
3. Test account balance, authentication, transaction limits, and duplicate requests.
4. Use automated regression tests for previously detected defects.
5. Use random testing and stress testing to identify unexpected failures.

Conclusion: Combining ECP, BVA, and automated regression testing provides systematic boundary coverage and improves the reliability of fund transfers.

Note: The ₹100–₹50,000 range is an illustrative assumption because the question does not specify actual banking limits.

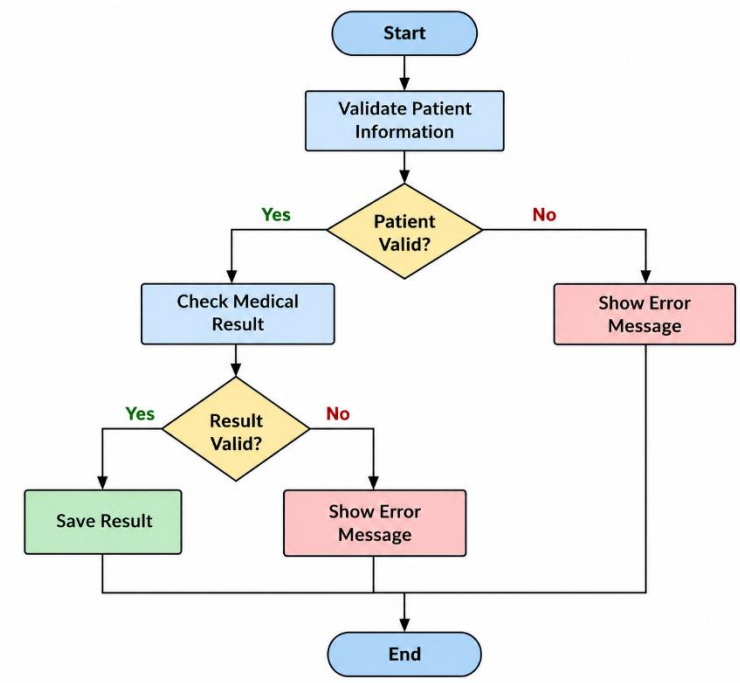
Question 2: Healthcare Application – Insufficient Test Coverage

1. Problem Analysis

Critical defects discovered after deployment indicate that the healthcare application was not tested adequately. The team may have tested only normal execution scenarios without checking all decision outcomes, error conditions, and alternative execution paths.

2. Conceptual Control Flow Graph (CFG)

Consider a simplified healthcare module that validates patient information and processes a medical result.



The graph contains decision nodes and alternative paths that must be tested.

3. Coverage Analysis

- Statement coverage: Ensures that every executable statement runs at least once. It may miss an untested decision outcome.
- Branch coverage: Ensures that each decision outcome, such as True and False, executes at least once.
- Path coverage: Tests execution paths through the control flow graph. Complete path coverage may be impractical when loops create many or infinitely many possible paths.

Statement coverage alone is insufficient because an error-handling branch may remain untested.

4. Improved Testing Strategy

1. Construct a CFG from the application logic.
2. Identify all decision points and independent paths.
3. Test valid patient details and invalid patient details.
4. Test valid medical results and invalid or missing results.
5. Verify database failures, invalid formats, and exception handling.
6. Measure statement and branch coverage and add tests for critical independent paths.
7. Perform regression testing after defect corrections.

5. Justification

Branch coverage checks alternative decision outcomes, while path-oriented testing explores important combinations of conditions. Additional validation and exception tests help identify failures before deployment.

Conclusion: A combination of statement coverage, branch coverage, independent-path testing, and healthcare-specific validation improves reliability and reduces the risk of critical post-deployment defects.

Question 3: Retail E-Commerce – Input Combination Errors

1. Problem Analysis

The e-commerce system accepts price, quantity, and discount values. Testing each input separately may not reveal errors caused by combinations, such as a high discount with a low price or an invalid quantity with a valid price.

2. Equivalence Class Partitioning (ECP)

Assume the system permits a positive price, quantity from 1 to 100, and discount from 0% to 50%.

Input	Valid Class	Invalid Classes
Price	Greater than ₹0	Zero or negative price
Quantity	1–100	Zero, negative, or above 100
Discount	0%–50%	Negative or above 50%

3. Boundary Value Analysis (BVA)

- Price: ₹0, ₹1, ₹2.
- Quantity: 0, 1, 2 and 99, 100, 101.
- Discount: -1%, 0%, 1% and 49%, 50%, 51%.

The expected result should be acceptance for valid values and appropriate validation errors for invalid values.

4. Decision Table Testing

Test Case	Price Valid	Quantity Valid	Discount Valid	Expected Result
TC1	Yes	Yes	Yes	Calculate total
TC2	No	Yes	Yes	Reject price
TC3	Yes	No	Yes	Reject quantity
TC4	Yes	Yes	No	Reject discount
TC5	No	No	Yes	Reject invalid inputs
TC6	Yes	No	No	Reject invalid inputs

The table can be extended to cover all relevant combinations and business rules.

5. Improved Testing Strategy

1. Identify valid and invalid equivalence classes.
2. Test boundaries for every input.
3. Apply decision table testing to combinations.
4. Verify the calculation:

   $$\text{Final Amount} = (\text{Price} \times \text{Quantity}) \times (1 - \text{Discount}/100)$$

5. Check rounding, maximum order values, and error messages.
6. Automate the tests and include them in regression testing.

Conclusion: ECP and BVA identify invalid values and boundary defects, while decision table testing detects interaction errors. Together, they provide better coverage than testing individual inputs alone.

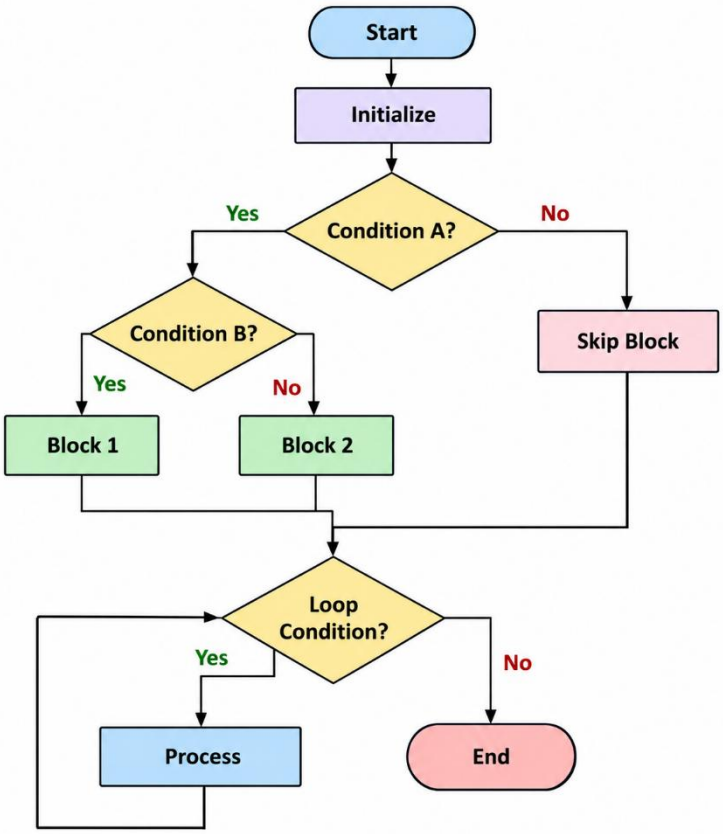
Question 4: Software Module with Nested Conditions and Loops

1. Problem Analysis

The software module contains nested conditions and loops, but only partial path coverage has been achieved. This may leave important combinations of decisions and loop conditions untested. Incorrect CFG construction can also cause independent execution paths to be missed.

2. Conceptual Control Flow Graph

Consider the following simplified logic:



3. Independent Paths

The important paths include:

1. Condition A is False.
2. Condition A is True and Condition B is True.
3. Condition A is True and Condition B is False.
4. The loop executes zero times.
5. The loop executes once.
6. The loop executes multiple times and terminates.

These paths must be adapted to the actual program logic.

4. Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths in a program.

$$V(G) = E - N + 2P$$

Where:

- E= number of edges.
- N = number of nodes.
- P = number of connected components, normally 1.

For a single connected graph, it can also be calculated as:

$$V(G) = D + 1$$

where D is the number of decision nodes.

5. Improved Testing Strategy

1. Draw the CFG from the source code.
2. Calculate cyclomatic complexity.
3. Identify a basis set of independent paths.
4. Test each path at least once.
5. Test loops with zero, one, two, and representative maximum iterations.
6. Test nested conditions using both True and False outcomes.
7. Use coverage tools and regression testing to verify the results.

Conclusion: Basis path testing improves systematic path coverage and helps reveal defects in nested conditions and loops. For loops, testing representative iteration counts is more practical than attempting every possible path.

Question 5: Financial Application – Hybrid Black-Box and White-Box Testing

1. Problem Analysis

The financial application uses both black-box and white-box testing, but defects continue to appear in production. This suggests that the methods may not be integrated effectively, important business rules may be missing from the test cases, or the coverage metrics may not reflect the actual risk.

2. Evaluation of Existing Testing

Black-box testing verifies the application's external behaviour but may miss untested internal execution paths. White-box testing examines code structure but may fail to identify incorrect business requirements if the expected behaviour is misunderstood.

Coverage percentages alone do not prove that all important financial scenarios have been tested.

3. Improved Hybrid Testing Strategy

Black-box testing

- Apply ECP to transaction amounts, account balances, and transaction types.
- Apply BVA to minimum and maximum transfer amounts.
- Test invalid accounts, insufficient funds, and duplicate transactions.
- Verify the displayed balance and transaction receipt.

White-box testing

- Construct a CFG for transaction validation and processing.
- Test independent paths for successful and failed transfers.
- Verify exception-handling and rollback paths.
- Check branches for sufficient and insufficient balances.
- Review code coverage and add tests for critical logic.

4. Example Test Cases

Test Case	Scenario	Expected Result
TC1	Valid transfer with sufficient balance	Transfer succeeds
TC2	Transfer amount exceeds limit	Transaction rejected
TC3	Insufficient account balance	Transaction rejected
TC4	Invalid beneficiary account	Validation error
TC5	Database failure during transfer	Transaction safely rolled back
TC6	Duplicate transfer request	No unintended duplicate debit

5. Test Adequacy Criteria

The team should measure statement coverage, branch coverage, and selected independent-path coverage. It should also verify requirements coverage and maintain regression tests for every fixed defect.

Conclusion: Integrating ECP, BVA, CFG-based path testing, and risk-based regression testing improves both input validation and internal logic verification, reducing the chance of financial defects reaching production.

Question 6: HR Payroll – Incorrect Bonuses at 5, 10, and 15 Years

1. Problem Analysis

The payroll system calculates bonuses based on years of service and performance ratings. Incorrect payouts at exactly 5, 10, and 15 years suggest boundary-condition errors, such as incorrect comparison operators or overlapping bonus categories.

For example, a condition using greater than 5 instead of greater than or equal to 5 may incorrectly exclude employees with exactly five years of service.

2. Boundary Value Analysis

Assume bonus categories change at 5, 10, and 15 years.

Boundary	Test Values (Years)	Purpose
5 years	4, 5, 6	Verify first threshold
10 years	9, 10, 11	Verify second threshold
15 years	14, 15, 16	Verify third threshold

The expected bonus must be determined from the official payroll rules.

3. Equivalence Class Partitioning

Equivalence Class	Example Input	Expected Behaviour
Below 5 years	3	Apply applicable lower-tier bonus
Exactly 5 years	5	Apply the correct 5-year rule
Between 5 and 10 years	7	Apply the applicable middle-tier bonus
Exactly 10 years	10	Apply the correct 10-year rule
Between 10 and 15 years	12	Apply the applicable higher-tier bonus
Exactly 15 years	15	Apply the correct 15-year rule
Above 15 years	16	Apply the applicable highest-tier rule
Invalid service period	-1	Reject invalid input

4. Performance Rating Tests

For every service category, test each valid performance rating and invalid ratings. This helps detect interaction errors between service duration and performance-based bonus calculations.

5. Why ECP and BVA Are Appropriate

Exploratory testing may identify unexpected defects, but it does not guarantee systematic testing of every threshold. ECP reduces redundant tests, while BVA specifically targets the values where classification errors are most likely.

Conclusion: ECP and BVA provide a structured and repeatable approach to validating payroll thresholds. Automated regression tests should ensure that employees receive the correct bonus after future code changes.

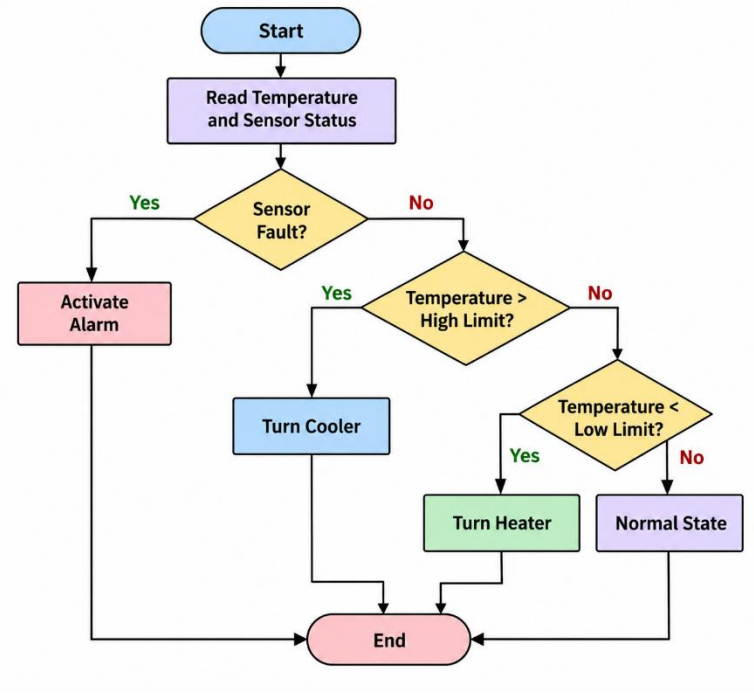
Question 7: Laboratory Temperature Control – MC/DC Testing

1. Problem Analysis

The laboratory controller uses if-else logic to control heaters, coolers, and alarms. The alarm failure during a sensor-malfunction sequence suggests that some combinations of conditions or decision outcomes were not tested.

2. Conceptual Control Flow Graph

Consider the following illustrative logic:



The graph must be adapted to the actual controller logic and the required safety response.

3. Branch Coverage Analysis

Branch coverage requires every decision outcome to execute at least once. However, achieving 100% branch coverage does not necessarily show that each individual condition independently affects the overall decision.

For example, testing a normal sensor reading and a sensor fault separately may still miss a fault occurring simultaneously with an extreme temperature.

4. Modified Condition/Decision Coverage (MC/DC)

MC/DC requires each individual Boolean condition to be shown to independently affect the outcome of its decision while the other relevant conditions are controlled.

For an illustrative alarm decision:

Alarm = SensorFault OR HighTemperature

Test	Sensor Fault	High Temperature	Alarm
TC1	False	False	False
TC2	True	False	True
TC3	False	True	True

- TC1 and TC2 demonstrate the independent effect of Sensor Fault.
- TC1 and TC3 demonstrate the independent effect of High Temperature.

This illustrates MC/DC for a two-condition OR decision. The complete controller may require additional tests for its other decisions.

5. Improved Testing Strategy

1. Identify every decision and Boolean condition.
2. Construct the CFG and identify all branches.
3. Design test pairs demonstrating each condition's independent effect.
4. Test sensor failure, disconnected sensors, high temperature, low temperature, and conflicting readings.
5. Verify that the alarm and safe-state mechanisms operate correctly.
6. Use automated coverage tools to verify MC/DC coverage.
7. Conduct fault-injection and hardware-in-the-loop testing where appropriate.

Conclusion: MC/DC testing provides stronger evidence of decision-logic correctness than statement or branch coverage alone. It is particularly important for safety-critical systems where an untested condition could prevent an alarm from activating.

Question 8: Travel Booking Portal – Promo Code Combinations

1. Problem Analysis

The travel booking portal allows multiple promo codes, but some combinations produce a negative total amount. Testing each promo code independently cannot identify all interaction defects. The system must validate promo eligibility, discount limits, user tiers, and the final payable amount.

2. Decision Table Testing

Assume the following illustrative rules:

- The booking price must be greater than zero.
- A promo code must be valid and eligible for the user tier.
- Combined discounts cannot exceed the booking price.
- The final amount cannot be negative.

Test Case	Price Valid	Promo Valid	User Eligible	Combined Discount Within Price	Expected Result
TC1	Yes	Yes	Yes	Yes	Accept booking
TC2	No	Yes	Yes	Yes	Reject invalid price
TC3	Yes	No	Yes	Yes	Reject or ignore invalid promo according to requirements
TC4	Yes	Yes	No	Yes	Reject or ignore ineligible promo according to requirements
TC5	Yes	Yes	Yes	No	Prevent negative total
TC6	Yes	Yes	Yes	Yes, with multiple eligible promos	Calculate combined discount correctly

3. Cause-Effect Analysis

The main causes are:

- C1: Booking price is valid.
- C2: Promo codes are valid.
- C3: User tier is eligible.
- C4: Combined discount does not exceed the price.

The desired effects are successful booking, appropriate validation errors, and a valid final amount. These causes and effects can be converted into a decision table to identify important combinations.

4. Optimized Testing Strategy

1. Apply ECP to price ranges, promo types, and user tiers.
2. Apply BVA to zero price, minimum price, and discount limits.
3. Test single and multiple promo codes.
4. Test expired, duplicated, and incompatible promo codes.
5. Verify the final amount using:

   Final Amount = Booking Price − Total Applicable Discount

6. Test cases where the combined discount equals or exceeds the booking price.
7. Use pairwise testing to reduce redundant combinations where appropriate, while retaining dedicated tests for high-risk rules.

Conclusion: Decision table testing and cause-effect analysis systematically cover interactions between price, promo codes, and user tiers. They help prevent negative totals while keeping the test suite efficient.

Question 9: Database Migration Tool – Infinite Loop Failures

1. Problem Analysis

The database migration tool reports high statement coverage but still experiences infinite-loop crashes. This shows that executing every statement once is insufficient to verify loop termination and iteration behaviour.

The defects may result from incorrect loop conditions, improperly updated counters, unexpected database records, or termination conditions that never become true.

2. Why Statement Coverage Is Insufficient

Statement coverage proves that statements have executed, but it does not guarantee that loops terminate correctly under all relevant conditions. A loop may execute successfully for a small dataset but fail when the dataset is empty, unusually large, or contains invalid records.

3. Basis Path Testing

The team should construct a CFG and identify independent paths through the migration logic.

For a loop that processes database records, test:

1. An empty dataset, where the loop executes zero times.
2. A dataset with one record.
3. A dataset with multiple records.
4. A dataset containing invalid records.
5. A dataset that triggers an error during processing.
6. A dataset large enough to test performance and termination.

4. Cyclomatic Complexity

The cyclomatic complexity is calculated using:

$$V(G) = E - N + 2P$$

For a single connected graph:

$$V(G) = D + 1$$

where D is the number of decision nodes.

If a simplified module contains three decision nodes, its cyclomatic complexity is:

$$V(G) = 3 + 1 = 4$$

Therefore, four independent paths form a basis set for that simplified module. The actual number must be calculated from the migration tool's CFG.

5. Improved Testing Strategy

- Identify all loop entry and exit conditions.
- Test zero, one, two, and representative maximum iterations.
- Verify that loop counters and record pointers are updated correctly.
- Test database errors and malformed records.
- Add timeouts or iteration limits where appropriate.
- Monitor loop termination, memory use, and execution time.
- Perform regression testing after fixing defects.

Conclusion: Basis path testing combined with loop-boundary testing provides stronger evidence of correct termination than statement coverage alone. Cyclomatic complexity helps estimate the minimum number of independent paths to examine.

Question 10: Food Delivery App – State Transition Testing

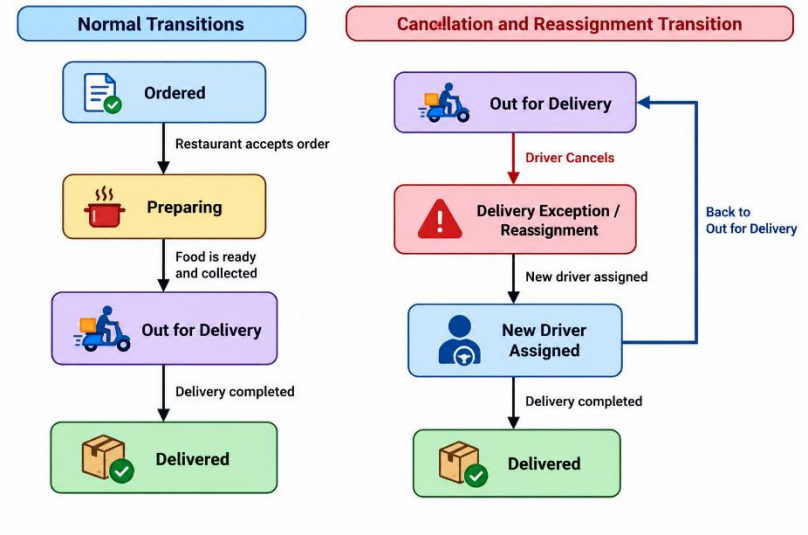
1. Problem Analysis

The food delivery application contains the states Ordered, Preparing, Out for Delivery, and Delivered. The application becomes stuck when the driver cancels during delivery, indicating that cancellation transitions and recovery behaviour may not have been tested.

ECP and BVA are useful for input values, but they do not adequately verify the sequence of states and events in a dynamic workflow.

2. State Transition Model

The normal transitions are:



The system should also define what happens if reassignment fails. For example, the order may be cancelled and a refund initiated according to the business rules.

3. State Transition Test Cases

Test Case	Initial State	Event	Expected State
TC1	Ordered	Restaurant accepts order	Preparing
TC2	Preparing	Food is ready and collected	Out for Delivery
TC3	Out for Delivery	Delivery completed	Delivered
TC4	Out for Delivery	Driver cancels	Delivery Exception
TC5	Delivery Exception	New driver assigned	Out for Delivery
TC6	Delivered	Driver cancellation event received	Remain Delivered; reject invalid transition
TC7	Delivery Exception	Reassignment fails	Apply defined cancellation or recovery process

4. Improved Testing Strategy

1. Identify all valid states and events.
2. Document valid and invalid state transitions.
3. Test normal delivery and cancellation scenarios.
4. Test repeated cancellation events and delayed notifications.
5. Verify that the user interface and backend show consistent states.
6. Test recovery, reassignment, and refund behaviour.
7. Automate transition tests and include them in regression testing.

Conclusion: State transition testing detects workflow defects that may not be revealed by input partitioning or boundary testing. It ensures that the application responds correctly to cancellation and recovery events.

Question 11: Cybersecurity Authentication – Brute-Force Vulnerabilities

1. Problem Analysis

The authentication module uses password strength, OTP timing, and IP location, yet brute-force vulnerabilities remain. This indicates that testing may have focused on normal login behaviour without sufficiently examining repeated failures, security controls, and combinations of authentication conditions.

2. Evaluation of Existing Testing

Black-box testing can verify login responses and error messages but may overlook internal control-flow defects. White-box testing can inspect authentication branches but may not identify every externally observable security weakness.

Therefore, both methods must be integrated with security-focused negative testing.

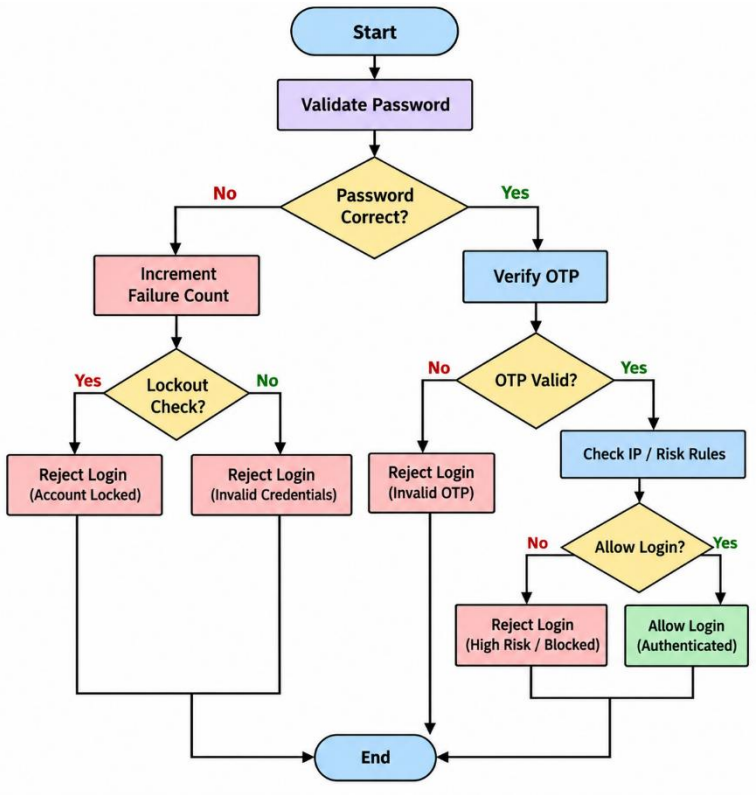
3. Hybrid Testing Strategy

Black-box testing with error guessing

- Submit incorrect passwords repeatedly.
- Enter incorrect, expired, and reused OTPs.
- Test missing and malformed authentication inputs.
- Test login attempts from different IP locations.
- Check account lockout, rate limiting, and session expiry.
- Verify that error messages do not reveal sensitive account information.

White-box testing with path analysis

Construct a CFG for the authentication logic, including:



The graph is conceptual and should be adapted to the actual implementation.

4. Improved Security Testing

1. Identify independent paths through password, OTP, and IP validation.
2. Test both successful and unsuccessful decision outcomes.
3. Verify rate limits and progressive delays.
4. Test lockout and recovery mechanisms.
5. Confirm that OTPs expire and cannot be reused.
6. Test concurrent requests and repeated attempts.
7. Use automated security tests and regression tests for every vulnerability found.

Conclusion: Error guessing helps uncover practical attack scenarios, while CFG-based path analysis verifies internal authentication logic. Combining both improves security coverage and reduces the risk of brute-force attacks.

Question 12: E-Learning Platform – Subscription Expiration and Upgrade Errors

1. Problem Analysis

The e-learning platform provides Free, Basic, and Premium subscription tiers. Errors occur when users upgrade exactly on the expiration date. This suggests inadequate testing of date boundaries, subscription validity, and transitions between subscription types.

2. Equivalence Class Partitioning

Assume the system distinguishes between active, expired, and not-yet-started subscriptions.

Equivalence Class	Example Input	Expected Result
Before expiration	One day before expiry	Existing access remains valid
On expiration date	Exact expiry date	Apply the documented expiration rule
After expiration	One day after expiry	Apply expired-subscription rules
Before subscription starts	Future start date	Access follows the defined start-date policy
Valid active subscription	Within the subscription period	Grant permitted access
Invalid subscription date	End date earlier than start date	Reject invalid dates

3. Subscription-Type Testing

Test each subscription type: Free, Basic, and Premium. Verify the courses accessible under each tier and ensure that an upgrade grants the correct permissions.

Important scenarios include:

- Free to Basic upgrade.
- Basic to Premium upgrade.
- Upgrade before expiration.
- Upgrade on the expiration date.
- Upgrade after expiration.
- Expired subscription renewal.
- Invalid or missing subscription dates.

4. Improved Testing Strategy

1. Partition subscription dates into valid and invalid classes.
2. Test dates immediately before, on, and after expiration.
3. Verify time-zone and date-format handling where applicable.
4. Test all subscription types and important transitions.
5. Confirm that access permissions update correctly after upgrades.
6. Verify payment failures, duplicate upgrade requests, and renewal behaviour.
7. Automate regression tests for date-based rules.

Conclusion: ECP provides representative coverage of subscription states, while boundary-focused tests target expiration-date errors. Together, they improve the accuracy of subscription transitions and prevent incorrect access permissions.

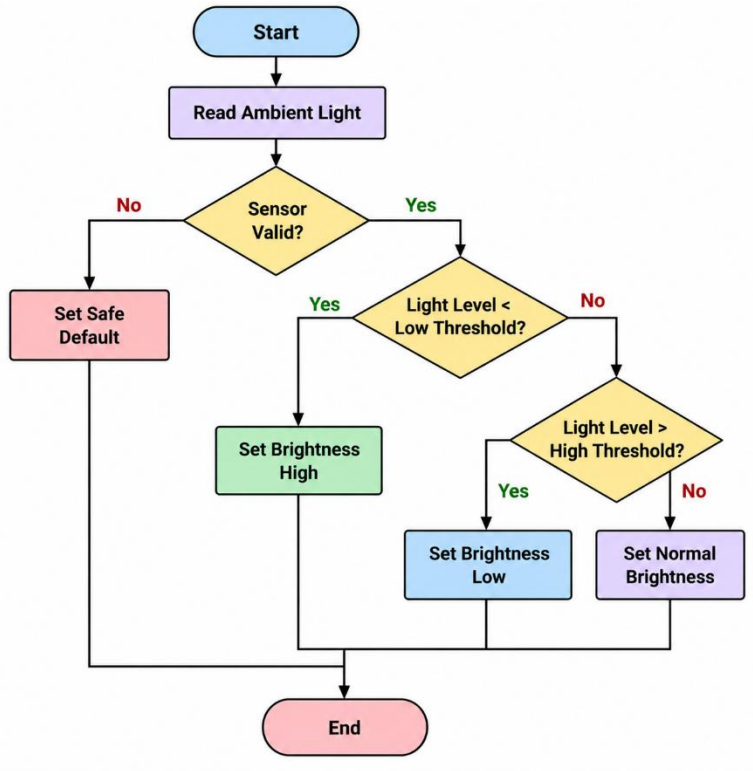
Question 13: Smart-Home Lighting Controller – Branch and Condition Coverage

1. Problem Analysis

The lighting controller adjusts brightness using ambient light sensors, switch-case statements, and Boolean expressions. Relying only on statement coverage may leave decision outcomes and individual Boolean conditions untested, causing incorrect brightness adjustments or hardware-control failures.

2. Conceptual Control Flow Graph

Consider this simplified lighting logic:



The CFG identifies decision points and alternative execution branches.

3. Limitations of Statement Coverage

Statement coverage checks whether executable statements have run at least once. It does not guarantee that:

- Every True and False decision outcome has been exercised.
- Every individual Boolean condition has affected a decision.
- Every switch-case branch has been tested.
- Invalid sensor values and hardware failures are handled correctly.

4. Branch and Condition Coverage

Branch coverage requires testing every decision outcome.

Condition coverage requires each individual Boolean condition to evaluate to both True and False.

For example, consider:

AdjustBrightness = SensorValid AND LowLight

Test Case	Sensor Valid	Low Light	Adjust Brightness
TC1	True	True	True
TC2	True	False	False
TC3	False	True	False
TC4	False	False	False

These cases exercise both outcomes of each condition and the overall decision.

5. Improved Testing Strategy

1. Construct the CFG from the actual source code.
2. Test every branch and switch-case option.
3. Ensure each Boolean condition evaluates to True and False.
4. Test sensor disconnection, invalid readings, and threshold values.
5. Verify brightness output at low, normal, and high ambient light levels.
6. Perform integration testing with the actual lighting hardware.
7. Automate regression tests and check coverage reports.

Conclusion: Branch and condition coverage provide stronger verification than statement coverage alone. They help identify missing logical cases and reduce the risk of incorrect interaction between software and lighting hardware.

Question 14: Logistics System – Robust Boundary Value Analysis

1. Problem Analysis

The logistics system calculates shipping costs using weight, destination zone, and delivery speed. Errors occurred because zero and negative weights were not tested adequately. The existing test design likely focused on normal positive values without considering invalid boundaries.

2. Robust Boundary Value Analysis

Assume the accepted shipping weight is from 0.1 kg to 100 kg. These limits are illustrative because the assessment does not specify actual limits.

Test Case	Weight	Expected Result
TC1	-0.1 kg	Reject invalid weight
TC2	0 kg	Reject zero weight
TC3	0.1 kg	Accept minimum valid weight
TC4	0.2 kg	Accept valid weight
TC5	99.9 kg	Accept valid weight
TC6	100 kg	Accept maximum valid weight
TC7	100.1 kg	Reject weight above maximum
TC8	Blank	Display validation error

TC9	Non-numeric input	Display validation error

3. Testing Other Inputs

Destination zones and delivery speeds must also be tested with valid and invalid values. For example, test a valid weight with an invalid destination zone and an unsupported delivery speed.

Additional tests should cover combinations such as minimum weight with express delivery and maximum weight with a distant destination.

4. Improved Testing Strategy

1. Define the permitted weight range.
2. Test values immediately below, at, and above both boundaries.
3. Test zero, negative, blank, and non-numeric values.
4. Verify shipping-cost calculations for different zones and delivery speeds.
5. Check that invalid inputs do not generate incorrect charges.
6. Add automated regression tests for all validation defects.

5. Importance of Negative Testing

Negative testing verifies that the application rejects invalid inputs safely. Without it, the system may calculate negative charges, accept impossible shipment weights, or produce inconsistent invoices.

Conclusion: Robust BVA tests both valid boundaries and invalid values outside the permitted range. It improves input validation, prevents incorrect shipping charges, and increases system reliability.

Question 15: Medical Imaging Software – Basis Path Testing

1. Problem Analysis

The medical imaging software processes large datasets using parallel loops. Inconsistent results occur when the dataset size is a prime number. The team's reliance on random testing is insufficient because random inputs may not consistently expose defects related to dataset size, parallel execution, or loop partitioning.

2. Evaluation of Random Testing

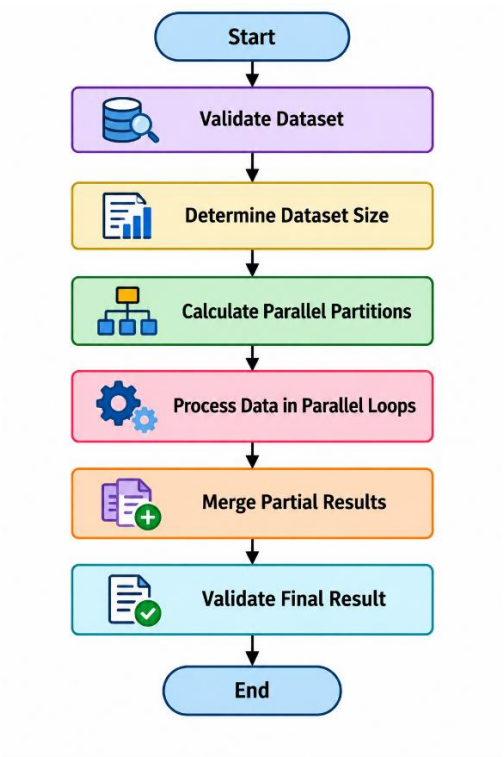
Random testing is useful for discovering unexpected failures, but it cannot guarantee coverage of prime-sized datasets or important loop conditions. It should therefore be retained as supplementary testing rather than used as the primary strategy.

3. White-Box Testing Strategy

The team should inspect the source code and construct a CFG covering:

- Dataset validation.
- Calculation of parallel partitions.
- Distribution of data across processing threads.
- Loop execution and termination.
- Merging of partial results.
- Error handling and output generation.

A conceptual flow is:



4. Basis Path Testing

Basis path testing identifies a set of linearly independent execution paths through the CFG.

Cyclomatic complexity can be calculated using:

$$V(G) = E - N + 2P$$

For a single connected graph:

$$V(G) = D + 1$$

where D represents the number of decision nodes.

The actual CFG must be examined to determine the number of independent paths and construct a suitable basis set.

5. Test Dataset Design

Test Case	Dataset Size	Purpose
TC1	0	Verify empty-dataset handling
TC2	1	Verify minimum dataset processing
TC3	7	Test a small prime-sized dataset
TC4	8	Compare with a nearby composite size
TC5	13	Test another prime-sized dataset
TC6	14	Compare with a nearby composite size
TC7	Large prime size	Test parallel partitioning and merging
TC8	Large composite size	Compare performance and correctness

For each dataset, verify that the parallel output matches a trusted sequential implementation or an independently calculated expected result.

6. Improved Testing Strategy

1. Construct the CFG and identify independent paths.
2. Calculate cyclomatic complexity.
3. Test loop entry, termination, and partition boundaries.
4. Compare prime and composite dataset sizes.
5. Test cases where the number of data elements is not evenly divisible among processing threads.
6. Verify numerical accuracy and deterministic output where required.
7. Use race-condition detection, stress testing, and regression testing.
8. Retain random testing for additional defect discovery.

7. Justification

Basis path testing provides systematic coverage of important control-flow paths. Testing prime and neighbouring composite dataset sizes directly targets the reported failure pattern. Comparing parallel and sequential results helps identify incorrect partitioning, missing data, duplicate processing, and result-merging defects.

Conclusion: Combining basis path testing, loop-boundary testing, and prime-sized dataset tests provides a structured approach to identifying defects in parallel medical imaging software. This improves mathematical accuracy, stability, and confidence in processing large datasets.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification